## Investigação de atividades pós-comprometimento: Analisar escaladas de privilégios e persistência

Acessar uma conta é apenas o primeiro passo. Uma vez que um invasor conquista acesso, sua próxima prioridade é dupla: expandir seu acesso e garantir que possa mantê-lo mesmo se a conta comprometida for bloqueada ou sua senha for redefinida.

Esta tarefa apresenta as ações pós-comprometimento mais comuns que um invasor realiza no Entra ID.

O foco aqui é o reconhecimento, saber o que procurar nos logs de auditoria.
Técnicas comuns pós-comprometimento.
Assim que o invasor é autenticado e começa a realizar ações dentro do tenant, as evidências relevantes são movidas para os logs de auditoria.
Os logs de auditoria capturam ações administrativas e quaisquer alterações no estado do tenant.

Atribuições de função, criação de usuários, modificações de políticas. Todas as alterações são registradas lá. Quando você suspeita que um usuário foi comprometido, o principal campo de filtragem é activityDisplayName, que indica a ação que o usuário realizou.
Para filtrar todos os registros de auditoria realizados em um locatário, irei usar a seguinte consulta Splunk:

**Listar todos os registros de auditoria:**

```
index="task-5" sourcetype="azure:aad:audit"
```

Resumidamente, os principais campos aos quais devemos prestar atenção em uma investigação de atividades pós-comprometimento são:

activityDisplayName: A atividade ou ação detalhada realizada por um usuário ou aplicativo.

initiatedBy: A conta ou aplicativo que realizou a ação.

targetResources: A conta ou os objetos que foram alterados ou afetados por uma ação.

Esses campos respondem à pergunta mais importante para analisar a atividade pós-comprometimento: o que foi alterado (activityDisplayName), quem fez essa alteração (initiatedBy) e os detalhes das alterações (targetResources).

**Atribuição de função**

Este é o caminho mais direto para obter acesso elevado: atribua uma função de ID Entra privilegiada à conta comprometida (ou a uma nova conta criada pelo invasor). Abaixo estão algumas funções de destino comuns:

Administrador Global: Controle total sobre o locatário.

Administrador do Exchange: Acesso a todas as caixas de correio.

Administrador de Usuários: Pode redefinir senhas e modificar contas.

Administrador de Aplicativos: Pode gerenciar registros de aplicativos e concessões de consentimento.

Uma atribuição de função legítima não é inerentemente suspeita. O que é suspeito é uma atribuição de função que ocorre fora dos fluxos de trabalho normais de provisionamento. Por exemplo, em um horário incomum, iniciada por uma conta que normalmente não executa essas ações, ou direcionada a uma conta que esteve recentemente envolvida em atividades de login suspeitas.

Usarei a consulta abaixo para listar todas as atividades de função atribuídas em um locatário. Explorar o campo `targetResources` para ver qual função foi adicionada a um usuário:

**Listar todas as atividades atribuídas à sua função:**

```
index="task-5" sourcetype="azure:aad:audit" activityDisplayName="Add member to role"
| table _time, activityDisplayName, initiatedBy.user.userPrincipalName, targetResources{}.userPrincipalName, targetResources{}.modifiedProperties{}.newValue | sort - _time
```

**Criação de Contas de Acesso Remoto**

Uma nova conta de administrador criada fora dos fluxos normais de provisionamento de RH/TI é um exemplo clássico de mecanismo de persistência. O invasor a cria, atribui a ela uma função privilegiada e a utiliza como alternativa caso a conta original comprometida seja recuperada.

Usarei a consulta Splunk abaixo para listar todas as contas criadas em um tenant:

```
index="task-5" sourcetype="azure:aad:audit" activityDisplayName="Add user"
| eval
initiator=coalesce('initiatedBy.user.userPrincipalName','initiatedBy.app.displayName')
| eval userCreated='targetResources{}.userPrincipalName'
| table _time, activityDisplayName,initiator, userCreated
```

**Adicionando Métodos Alternativos de MFA**

Se um invasor registrar seu próprio aplicativo autenticador ou número de telefone na conta comprometida, ele manterá o acesso mesmo após a vítima alterar sua senha, pois controla o fator de MFA. Isso aparece nos logs de auditoria como um evento de registro de MFA em um objeto de usuário, iniciado pelo próprio usuário.

Ao integrar um dispositivo de MFA, vários tipos de logs podem ser gerados para cada etapa. Para validar se um usuário tentou adicionar um dispositivo de MFA à sua conta, irei usar a consulta abaixo:

```
index="task-5" sourcetype="azure:aad:audit" activityDisplayName="User started
security info registration" loggedByService="Authentication Methods"
operationType="Add"
| eval
initiator=coalesce('initiatedBy.user.userPrincipalName','initiatedBy.app.displayName')
| table _time, activityDisplayName, initiator, initiatedBy.user.ipAddress,
additionalDetails{}.value
```

Eu posso explorar o campo additionalDetails para verificar o tipo de dispositivo que o usuário adicionou.

## Prática

Irei investigar para identificar a atividade pós-comprometimento na conta comprometida usando os logs do Splunk.

**Descobertas:**

**Endereço de e-mail do usuário que o invasor criou:**
Usei essa pesquisa para listar todas as atividades atribuídas:

index="task-5" sourcetype="azure:aad:audit" activityDisplayName="Add member to role"
| table _time, activityDisplayName, initiatedBy.user.userPrincipalName, targetResources{}.userPrincipalName, targetResources{}.modifiedProperties{}.newValue | sort - _time

Descobri que o email criado foi: rafael.maciel@finegalo.thm

**Qual função (Role.DisplayName) foi atribuída a essa nova conta**
Clicando em activityDisplayName e visualizar eventos descobri que:

```
{ [-]
  displayName: Role.DisplayName
  newValue: "Global Administrator"
  oldValue: null
}
```

A função atribuída foi: Global Administrator

**Quando o invasor adicionou um novo dispositivo MFA**
Usei essa consulta:

```
index="task-5" sourcetype="azure:aad:audit" activityDisplayName="User started security info registration" loggedByService="Authentication Methods" operationType="Add"
| eval initiator=coalesce('initiatedBy.user.userPrincipalName','initiatedBy.app.displayName')
| table _time, activityDisplayName, initiator, initiatedBy.user.ipAddress, additionalDetails{}.value
```

Events Patterns **Statistics (1)** Visualization

Show: 20 Per Page ✓ Format ✓ Preview: On

| _time | activityDisplayName | initiator | initiatedBy.user.ipAddress | additionalDetails{}.value |
|---|---|---|---|---|
| 2026-03-04 13:36:58 | User started security info registration | rafael.maciel@finegalo.thm | 149.102.234.27 | 0066e657-5110-4705-bf57-ec662664d38d<br>AppCodeOnly<br>Authenticator App with Code |

O resultado foi esse: 2026-03-04 13:36

Autor: Paulo Cesar
Formado em Segurança da informação
linkedin.com/in/paulo-cesar-security